

《网络数据采集实践》

课程设计

题 目	Python 图书信息采集与分析项目
学院（系）	信息技术学院
专 业	软件工程
班 级	2023 级创新 1 班
姓 名	组长姓名*
	成员一
	成员二
	成员三
	成员四
指导 教师	伦金龙

摘要

本项目设计并实现了一套基于 Python 的图书信息采集与分析系统，旨在通过网络爬虫技术从当当网采集 Python 相关图书的详细信息，包括图书标题、价格、作者、出版社、出版日期、详细描述及店铺名称等。系统采用分层架构设计，包括数据采集、存储、处理、分析和可视化模块，使用了 Urllib、BeautifulSoup、Flume、HDFS、Pandas、Matplotlib 等工具和技术。通过定时采集和断点续采机制确保数据的时效性和完整性，通过分布式存储和多维度分析生成价格分布、畅销作者排名、出版社数量分布等可视化图表，为用户提供了全面的市场洞察。

在数据采集模块中，系统通过模拟浏览器请求和动态延时策略，高效获取网页源代码，并利用 Flume 将数据实时传输至 HDFS，确保数据存储的可靠性和可扩展性。在数据处理模块中，系统结合 BeautifulSoup 提取关键字段，并通过 Pandas 进行数据清洗和格式标准化，提升数据质量。在数据分析模块中，系统利用 Pandas 和 Matplotlib 生成多维度的可视化图表，包括价格分布直方图、畅销作者排名条形图、出版社数量分布图、销售热词词云等，帮助用户快速了解市场动态。

项目成果验证了数据采集与分析技术在图书市场的应用价值，为读者降低了选书成本，为作者和出版商提供了优化选题和定价策略的依据。未来，系统计划扩展数据源至更多平台（如京东、亚马逊），并引入时间序列分析等预测模型，进一步提升系统的前瞻性和商业价值，同时探索 SaaS 化服务模式，支持企业定制化数据订阅。

【关键词】图书信息、数据采集、数据分析、可视化、大数据（3-5 个、小四、黑体）

ABSTRACT

外文摘要内容与中文摘要对应：（小四、Time New Roman 字体，1.5 倍行距）

This project designs and implements a Python-based book information collection and analysis system aimed at collecting detailed information about Python-related books from Dangdang.com using web scraping technology. The system adopts a layered architecture with modules for data collection, storage, processing, analysis, and visualization, leveraging tools such as Urllib, BeautifulSoup, Flume, HDFS, Pandas, and Matplotlib. By implementing scheduled data collection and breakpoint continuation mechanisms, the system ensures data timeliness and integrity, while distributed storage and multi-dimensional analysis generate visualizations like price distribution, best-selling author rankings, and publisher distribution charts, providing users with comprehensive market insights.

In the data collection module, the system efficiently retrieves web page source codes through simulated browser requests and dynamic delay strategies, and uses Flume to transmit data in real-time to HDFS for reliable and scalable storage. In the data processing module, key fields are extracted using BeautifulSoup and data quality is improved through Pandas for data cleaning and format standardization. In the data analysis module, multi-dimensional visualizations are generated using Pandas and Matplotlib, including price distribution histograms, best-selling author ranking bar charts, publisher distribution charts, and keyword word clouds, helping users quickly understand market dynamics.

The project outcomes validate the application value of data collection and analysis technologies in the book market, reducing book selection costs for readers and providing optimization references for authors and publishers in topic selection and pricing strategies. Future plans include expanding data sources to more platforms (such as JD.com and Amazon) and introducing predictive models like time series analysis to enhance the system's foresight and commercial value, while exploring SaaS service models to support customized data subscriptions for enterprises.

【Key Words】 Book Information, Data Collection, Data Analysis, Visualization, BigData Technology (3-5 个、小四、Time New Roman 字体)

目 录

第 1 章 概述	1
1.1 课题分析	1
1.2 课题要求	2
1.3 理论基础	2
1.4 预期结果	2
第 2 章 项目需求分析	3
2.1 项目背景与目标	3
2.2 项目数据来源	4
2.3 项目数据采集	5
2.4 项目数据存储	6
2.5 项目数据处理	6
2.6 项目数据分析与可视化	7
2.7 系统性能需求	8
2.8 技术需求	9
第 3 章 项目开发设计	11
3.1 系统架构设计	11
3.2 详细模块设计	12
3.2.1 数据采集模块	12
3.2.2 数据存储模块	12
3.2.3 数据处理模块	13
3.2.4 数据分析模块	15
3.2.5 数据可视化模块	15
3.3 数据定期采集与存储功能设计	15
第 4 章 项目开发实施	17
4.1 项目环境准备	17
4.1.1 生产环境操作系统搭建	17
4.1.2 Hadoop 大数据系统搭建	19
4.1.3 开发环境准备	21

4.2 功能模块开发	23
4.2.1 数据采集模块开发	23
4.2.2 数据存储模块开发	26
4.2.3 数据处理模块开发	28
4.2.4 数据分析模块开发	31
4.2.5 数据可视化模块开发	34
4.3 数据定期采集及存储功能部署	38
 第 5 章 项目测试实施	 41
5.1 数据采集功能测试	41
5.2 数据采集完整性测试	41
5.3 数据解析功能测试	42
5.4 数据可视化功能测试	42
5.5 数据定期采集及存储功能测试	42
 第 6 章 结论	 45
 参考文献	 47
 心得体会	 49
 致谢	 51

第 1 章 概述

1.1 课题分析

- ① 正文宋体小四号，正文段落和标题都是 1.5 倍行距，正文段落首行缩进 2 字符
- ② 文中图片，命名以章号开头，按顺序排列，如第 1 章第 1 个图为“图 1.1”。黑体五号加粗，居于图正下方。
- ③ 文中表标题和表序号居中置于表上方（黑体五号，数字和字母用 Times New Roman 五号粗体）；

图书信息的网络化是数字化时代的重要特征。随着电商平台（如亚马逊、当当网）和在线图书社区（如豆瓣读书、Goodreads）的兴起，图书信息的存储和传播逐渐从传统的纸质媒介转向网络平台。这些平台不仅提供了丰富的图书资源，还积累了大量用户评价、评分和阅读数据。这些数据对于图书市场分析、读者行为研究以及图书推荐系统的设计具有重要价值。

在数字化时代，数据是决策的基础。对于图书行业而言，网络数据采集与分析可以帮助出版商了解市场需求、优化图书选题策略，帮助电商平台提升用户体验、优化推荐算法，帮助学术研究者分析读者行为和阅读趋势。然而，这些数据通常以非结构化形式存储在网页中，需要通过数据采集技术将其转化为结构化数据，再进行清洗、分析和可视化，才能为实际应用提供支持。

《Python 图书信息采集与分析项目》旨在通过 Python 技术实现对网络图书信息的全面采集与深度分析，为读者、作者、出版商等多方提供精准的数据洞察。项目的核心目标是利用 Python 强大的网络爬虫和数据分析能力，采集各大图书平台（如亚马逊、当当网、豆瓣读书等）的 Python 相关图书数据，包括价格分布、出版社数量、畅销作者排名、图书榜单以及各渠道的销售情况等关键信息。通过多维度的数据分析，帮助用户快速了解 Python 图书市场的动态趋势。同时，项目还将深入挖掘图书描述信息中的文本内容，通过自然语言处理技术提取关键知识点和高频主题，揭示 Python 图书内容的核心知识体系。这不仅为读者提供了选书的科学依据，也为作者和出版商优化图书选题和内容设计提供了数据支持。

1.2 课题要求

本课程项目实施以下主要功能：

- (1) 定期采集 Python 图书信息并保存至 HDFS，参考网站 <https://www.dangdang.com/>；
- (2) Python 图书信息解析与预处理，提取关键信息，以预设格式保存至本地；
- (3) Python 图书信息处理与分析，完成数据预处理（归一化、缺失值处理），输出数据统计；
- (4) Python 图书信息可视化，制作数据分析报告（PPT，不少于 5 页内容）。

1.3 理论基础

本项目作为《数据采集与数据预处理》课程的实践实训环节，旨在深入运用数据采集与数据预处理的相关基础知识，全面实现数据采集与清洗、大数据存储与管理、数据解析与预处理、数据处理与分析以及撰写数据分析报告等一系列功能。在实施过程中，本项目主要依托强大的集成开发环境(IDE)工具——Pycharm，以高效、便捷地完成各项实践任务。

1.4 预期结果

本项目致力于构建一套完整的数据采集与分析系统，精准采集目标网页信息，高效解析并预处理数据，确保关键信息能够灵活存储至本地文件或数据库。系统支持对核心数据进行深度统计分析，并以专业数据分析报告的形式呈现 Python 图书的市场洞察，包括价格分布、出版社数量、畅销作者排名、图书榜单及各渠道销售情况等关键指标。

通过数据采集与可视化分析，本项目为 Python 图书的读者、作者及出版商提供全面的市场情报支持。同时，结合自然语言处理技术，系统能够提取文本中的关键知识点和高频主题，深入挖掘 Python 图书内容的核心知识体系，助力用户快速掌握领域重点。

第 2 章 项目需求分析

2.1 项目背景与目标

在当下，Python 技术凭借其简洁易用的语法、强大的功能以及广泛的应用领域，在编程语言市场中占据着愈发重要的地位。随着 Python 技术的广泛普及与深入应用，Python 图书市场也迎来了蓬勃发展的黄金时期，市场需求呈现出持续增长的态势。

为了帮助 Python 图书的读者、作者以及出版商能够更加全面、深入地了解市场动态，精准把握市场趋势，本项目致力于构建一套高效且智能的数据采集与分析系统。该系统将聚焦于当当网这一重要的图书销售平台，通过采集其上的 Python 图书信息，运用先进的数据处理技术进行数据清洗、多维度统计分析以及直观的可视化展示，最终定期为用户提供有价值的数据分析报告，助力各方在 Python 图书市场中做出更加明智的决策。

为实现上述项目目的，现细化项目目标如下：

- (1) **数据采集：**每日定时启动采集任务，精准获取当当网 Python 图书的网页源代码，确保所采集的数据既具备高度的时效性，又能完整涵盖所有关键信息，为后续的数据处理与分析提供坚实的基础。
- (2) **数据存储：**借助 Flume 这一高效的数据传输工具，将采集到的网页源代码实时或定时传输至 HDFS（Hadoop Distributed File System），充分利用 HDFS 的分布式存储特性，保障数据的可靠性和可扩展性，以应对不断增长的数据量。
- (3) **数据解析：**运用 Python 语言结合 BeautifulSoup 库，从复杂的网页源代码中精准提取关键信息，包括图书标题、价格、作者、出版社、出版日期、详细描述以及店铺名称等，为后续的数据分析提供结构化的数据支持。
- (4) **数据分析：**对提取到的结构化数据进行多维度的统计分析，涵盖价格分布、出版社数量统计、畅销作者排名、图书榜单生成、各渠道销售情况分析以及图书描述词云分析等多个方面，全面挖掘数据背后的潜在价值。
- (5) **数据可视化：**通过直观、美观的图表和详细的报告展示分析结果，为用

户提供有价值的数据分析报告,使用户能够轻松理解数据所传达的信息,快速获取有价值的市场洞。

2.2 项目数据来源

当当网作为国内重要的图书销售平台,图书类目相对较为全面,本项目将聚焦于该平台 <https://www.dangdang.com/>, 采集 Python 图书信息页面的网页源代码。



图 2.1 当当网主页

(1) 网站页面逻辑结构分析

网站页面逻辑结构分析,工作内容是分析目标网站基本布局、数据罗列的逻辑结构。Python 图书信息页面可以通过网站主页导航触达,通过观察页面发现网站单个页面展示 60 本图书信息,包含 100 个分页,共展示 6000 本图书信息,然后分析各个页面对应 URL <http://category.dangdang.com/pg1-cp01.54.06.19.00.00.html> 的规律发现,不同页面 URL 的区别是链接中关键字 pg 后面的数字,如 pg1 代表第 1 页、pg2 代表第 2 页,依次类推。



图 2.2 当当网 Python 图书信息页面

(2) 网站页面数据结构分析

网站页面数据结构分析，工作内容是分析页面中数据分布的具体位置，还有哪些关联页面保存着相关的数据。经观察发现当当网 Python 图书信息页面的关键数据主要集中在页面的中部，对应的标签是<ul class="bigimg" id="component_59">.....。

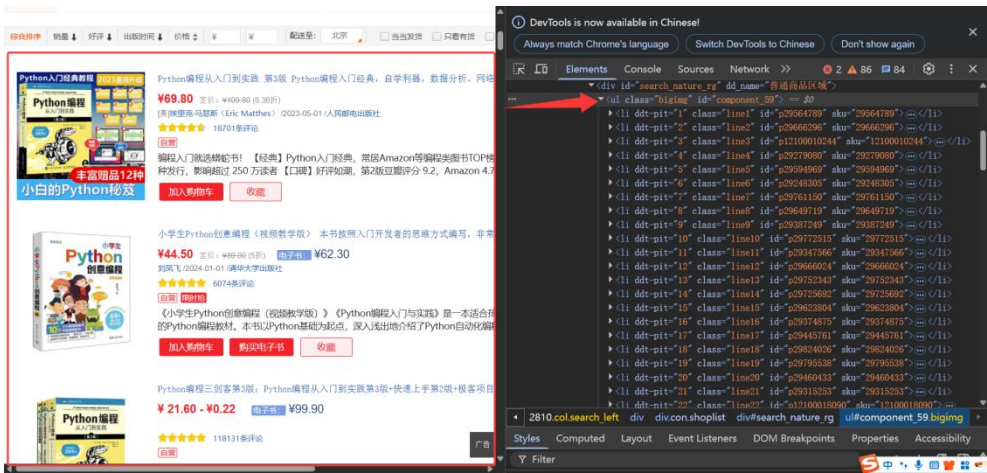


图 2.3 当当网 Python 图书信息页面数据分布

2.3 项目数据采集

(1) 采集目标

每日定时从当当网采集 Python 图书的网页源代码，确保采集内容完整无缺，涵盖所有与 Python 图书相关的关键信息，为后续的数据处理提供全面的数据基

础。

(2) 采集方式

采用 Python 爬虫技术，如 urllib 库或 scrapy 框架，高效获取网页源代码。针对网页的动态加载问题，必要时结合 Selenium 或 Playwright 等工具，模拟用户操作，确保能够完整采集到动态加载的内容。

(3) 采集频率

严格按照每日定时的方式进行采集，确保数据的时效性，及时反映市场动态变化。

(4) 采集效率

对爬虫性能进行深度优化，采用代理 IP 池或设置合理的请求间隔等方式，避免因频繁请求导致 IP 被封禁，保障采集任务的稳定进行。

(5) 数据合规性

严格遵守当当网的爬虫政策，如 robots.txt 文件的规定，确保采集行为合法合规。同时，避免采集敏感或受版权保护的内容，保护相关权益人的合法权益。

2.4 项目数据存储

(1) 存储方式

利用 Flume 将采集到的网页源代码实时或定时传输至 HDFS，确保 Flume 配置准确无误，能够高效、稳定地处理大规模数据，保障数据存储的及时性和可靠性。

(2) 存储结构

在 HDFS 中精心设计合理的存储路径和文件命名规则，例如按照日期进行分区存储，方便数据的管理和查询，提高数据处理的效率。

(3) 数据备份

定期对 HDFS 中的数据进行备份，制定完善的数据备份策略，确保数据的安全性，防止数据丢失或损坏。

2.5 项目数据处理

(1) 数据读取

使用 Python 的 `hdfs` 或 `pyarrow` 库等工具，从 HDFS 中高效读取网页源代码数据，为后续的数据解析提供便利。

(2) 数据解析

运用 `BeautifulSoup` 库从网页源代码中精准提取以下字段：图书标题、价格、作者、出版社、出版日期、详细描述以及店铺名称等，将非结构化的网页数据转化为结构化的数据。

(3) 数据清洗

对提取到的数据进行全面清洗，处理缺失值（如某些字段可能为空的情况），统一数据格式（如规范日期格式、统一价格单位），去除重复数据，提高数据的质量和可用性。

(4) 中分分词及清洗

针对图书详细描述得文本信息进行中文分词和清洗，通过自然语言处理技术提取关键知识点和高频主题，揭示 Python 图书内容的核心知识体系。

2.6 项目数据分析与可视化

(1) 统计分析

- 价格分布分析：统计不同价格区间的图书数量，深入分析价格趋势，为读者提供价格参考，为出版商和作者制定合理的价格策略提供依据。
- 出版社数量分析：统计不同出版社的图书数量，分析市场分布情况，帮助出版商了解自身在市场中的地位，为市场拓展提供参考。
- 畅销作者排名：根据销量或其他指标对作者进行排名，识别热门作者，为读者推荐优质作者的作品，为作者提升知名度和影响力提供支持。
- 各渠道销售情况：分析不同店铺的销售数据，识别销售热点，为店铺优化销售策略，为出版商选择合适的销售渠道提供依据。
- 图书销售热词词频分析：通过对图书详细描述文本进行深度词频统计，精准剖析图书的核心知识体系架构，为作者优化图书编撰策略提供极具价值的参考与方向指引。

(2) 图表可视化

使用 Python 可视化库，如 `Matplotlib`、`Seaborn`、`Plotly` 等，生成各种类

型的图表。通过柱状图、饼图、词云图等直观的方式展示分析结果，使用户能够更加清晰地理解数据所传达的信息。

图表类型

- 价格分布：使用直方图或箱线图展示不同价格区间的图书数量，直观呈现价格分布情况。
- 出版社数量：使用柱状图或饼图展示不同出版社的图书数量，清晰展示市场分布格局。
- 畅销作者排名：使用条形图展示作者排名，方便用户快速了解热门作者。
- 销售情况：使用热力图或饼图展示各渠道销售情况，帮助用户分析销售趋势。
- 词云图：展示图书描述中的高频关键词，揭示图书内容的核心主题。

展示方式

定期以 PPT 报告的形式呈现分析结果，助力用户及时、全面地掌握 Python 图书的相关信息，为决策提供有力支持。

2.7 系统性能需求

(1) 采集效率

对爬虫性能进行深度优化，确保每天定时采集任务能够按时、高效完成，避免因采集延迟导致数据时效性降低。

(2) 处理效率

运用分布式计算框架，如 MapReduce，对大规模数据进行高效处理，提高数据处理的速度和效率，确保系统能够及时响应用户需求。

(3) 存储效率

确保 HDFS 的读写性能满足系统需求，优化数据存储和读取策略，提高数据存储和访问的效率。

(4) 扩展性

系统需具备良好的扩展性，能够支持数据量的不断增长，从几千条数据到几百万条数据都能稳定运行。可考虑使用云存储（如 AWS S3）或分布式文件系统，

以应对未来数据量的扩张。

2.8 技术需求

(1) 开发语言

使用 Python 作为主要开发语言，充分利用 Python 丰富的库和强大的开发能力，提高开发效率和代码质量。

(2) 工具与框架

- 数据采集：urllib、Selenium、Scrapy 等，满足不同场景下的数据采集需求。
- 数据存储：Flume、HDFS，确保数据的高效存储和管理。
- 数据解析：BeautifulSoup、lxml，精准提取网页中的关键信息。
- 数据分析：Pandas、NumPy，进行高效的数据处理和分析。
- 可视化：Matplotlib、Seaborn、WordCloud 等，生成直观、美观的图表。
- 自然语言处理：jieba，进行文本分词和关键词提取。

(3) 部署环境

支持本地部署或云部署（如 AWS、阿里云等），根据用户需求和实际情况选择合适的部署方式，提高系统的灵活性和可用性。

第 3 章 项目开发设计

3.1 系统架构设计

根据上述项目需求分析结果，本项目将采用分层设计，包括数据采集、数据存储、数据处理与清洗、数据统计分析以及数据可视化。

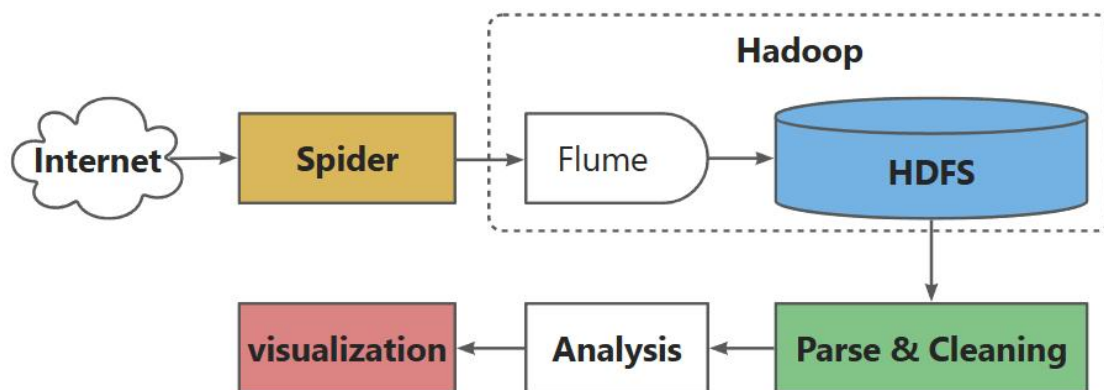


图 3.1 系统数据流程图

具体分为以下模块：

- 数据采集层：负责定时爬取当当网 Python 图书页面源码。
- 数据存储层：通过 Flume 将数据存入 HDFS，按日期分区存储。
- 数据处理层：解析、清洗网页数据，转换为结构化格式。
- 数据分析层：执行统计分析、文本挖掘。
- 可视化层：生成图表和报告。

技术栈

- 开发环境系统：Windows11
- 生产环境系统：Centos7.9
- 核心语言：Python 3.7+ (Anaconda3-5.3.1)
- 采集工具：Python 内置库 Urllib
- 存储工具：HDFS + Flume (数据流管理)
- 解析工具：BeautifulSoup、lxml
- 分析工具：Pandas、NumPy、jieba (分词)
- 可视化工具：Matplotlib、Seaborn、Plotly、WordCloud
- 部署环境：支持本地/云 (AWS、阿里云)

3.2 详细模块设计

基于本项目采用分层设计，则各层模块详细设计如下。

3.2.1 数据采集模块

1. 页面采集

网页源代码采集功能以类封装，遍历待爬取页面获取网页源代码，并将每个页面按行保存至本地文件。为防止采集过程中断导致数据缺失，需增加检查逻辑重新爬取。

```
1 class MySpider:
    def __init__(self):
        # 基于模拟浏览器头信息创建爬虫对象

    def getData(self, url):
        # 向指定网址 url 发送请求获得页面响应

    def writeData(self, resp):
        # 将网页响应结果 resp 写入本地文件

    def start(self):
        # 启动爬虫

    def isNew(self):
        # 判断是断点续爬（针对爬取过程中断情况）还是重新开始

    def updateList(self, i):
        # 更新爬取过程记录列表
```

2. 反爬策略：

- (1) 使用浏览器头信息 User-Agent；
- (2) 请求间隔优化, 设置动态延时（2-5 秒）。

3. 合规性：

限制并发请求量。

3.2.2 数据存储模块

1. HDFS 配置：文本文档中以行保存每个网页源代码。

- (1) 存储路径：/flume/dd_python_html/{YYYYMMDD}/（按日期分区）；

(2) 文件格式：文本文件（原始 HTML）。

2. Flume Agent 配置：

```
1 # 配置组件名称
  a1.sources = r1
  a1.channels = c1
  a1.sinks = k1

  # Source
  Source 组件配置

  # Channel
  Channel 组件配置

  # Sink
  Sink 组件配置

  # 组件绑定
```

3.2.3 数据处理模块

本项目数据处理模块包括网页源代码数据解析、结构化数据清洗和文本数据预处理。

1. 数据解析：

数据解析主要针对采集到的网页源代码进行解析，而本项目采集到的网页源代码保存在 HDFS，因此本项目数据解析思路如下。

(1) HDFS 网页源代码读取功能以类封装，遍历指定目录路径下文档，读取文档并返回文档内容。

```
1 # 文档内容遍历功能以类封装，遍历读取文档中的内容
  class LoadHDFSFiles:
      def __init__(self, HDFS_path, IP, port, userName):
          # 根据提供的 HDFS 信息创建 HDFS 客户端连接对象

      def __iter__(self):
          # 遍历指定路径 HDFS_path 下的文档，获取文本内容
```

(2) 网页源代码解析功能以类封装，通过调用 HDFS 读取文档功能（步骤 1），获取网页源代码，结合 BeautifulSoup 选择器提取目标数据并写入

本地文档。

```

1 # 网页源代码解析功能以类封装，调用 HDFS 读取文档功能获取网页源代码，
  # 结合 BeautifulSoup 选择器提取目标数据并写入本地文档
  class MyParser:
      def __init__(self, in_path, out_path):
          # 根据提供的输入路径和输出路径创建网页源代码解析对象

      def parse(self, content):
          # 基于读到的网页源代码 content，结合 BeautifulSoup 提取目
          # 标数据

      def writeData(self, rs_list):
          # 将所提取到的目标数据 rs_list 写入文本文档

      def start(self):
          # 启动解析功能

```

2. 数据清洗:

- (1) 缺失值处理：缺少评分和评论数的统一置为“0”，没有店铺名称信息的统一设为“自营”。
- (2) 格式标准化：出版社名称缺少关键字“出版社”，统一在后面填补“出版社”。

3. 文本数据预处理

针对图书详细描述得文本信息进行中文分词、清洗。

```

1 # 遍历文件功能以类封装，遍历指定目录路径下文档，读取文档并返回文档内
  # 容
  class LoadData:
      def __init__(self, inpath):
          # 根据输入文档路径 inpat 创建文档内容加载对象

      def __iter__(self):
          # 读取指定路径为 inpath 的文档，获取文档中每行文本内容并返回

  # 中文分词及清理以类封装，包括停用词文件路径初始化、加载停用词方法、
  # 中文文本分词及字词清洗方法
  class ClearWord:
      def __init__(self, stopWord_path):
          # 根据停用词文档路径 stopWord_path 创建词清洗对象

      def loadStopWord(self):

```

	<pre># 加载停用词方法 def clearWord(self, doc): # 中文分词及清洗：停用词、数字、单个字符、空字符、特殊字词</pre>
--	--

3.2.4 数据分析模块

本项目将从以下几方面展开 Python 图书信息分析：

- (1) 价格分布分析：统计不同价格区间的图书数量。
- (2) 出版社数量分析：统计不同出版社的图书数量。
- (3) 畅销作者排名：统计不同作者的图书销量。
- (4) 各渠道销售情况：统计自营与非自营的图书销量，统计不同店铺的图书销量。
- (5) 图书描述关键热词分析：统计 Python 图书描述字词的数量。

3.2.5 数据可视化模块

基于以上数据分析模块设计，本项目可视化模块包括如下内容：

- (1) Python 图书价格分布直方图。
- (2) Python 图书出版社数量分布直方图。
- (3) Python 图书作者销量直方图（前 10 名）。
- (4) Python 图书自营与非自营的店铺销量分布饼图和店铺销量直方图（前 10 名）。
- (5) Python 图书描述关键热词直方图和词云图。

3.3 数据定期采集与存储功能设计

基于需求分析，本项目采用每日定时采集机制，以保障数据的时效性与完整性。具体而言，数据采集程序将部署于服务器环境，通过预设的时间调度自动触发运行。为应对采集过程中可能出现的意外中断风险，系统特别设计了断点续采

机制，通过内置的智能校验逻辑自动识别数据缺失片段，并启动定向重爬程序完成数据补全。

在数据传输环节，部署 Flume Agent 作为实时数据监控中枢，对采集数据文件实施秒级动态监测。一旦检测到数据更新，系统将立即启动增量同步机制，通过高效的数据管道将最新数据实时传输至 HDFS 分布式存储系统。存储层采用日期维度进行精细化分区管理，通过构建时间序列索引体系，显著提升数据查询响应速度与检索精准度。

本方案通过三层保障机制（定时采集、断点续采、实时同步）与智能存储策略，构建起完整的数据采集-传输-存储闭环体系，确保数据资产的全链路质量控制。

第 4 章 项目开发实施

4.1 项目环境准备

为确保项目顺利开发及部署，现需完成生产环境操作系统搭建、Hadoop 大数据系统搭建以及开发环境准备。

4.1.1 生产环境操作系统搭建

为保证项目能够稳定运行，本项目生产环境操作系统选用 Centos7.9，具体如下。

```
[root@hadoop ~]# cat /etc/redhat-release
CentOS Linux release 7.9.2009 (Core)
```

图 4.1 主机操作系统

主机共有 2 个 CPU，每个 CPU 包含 2 核，具体信息如下。

```
[root@hadoop ~]# lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                4
On-line CPU(s) list:   0-3
Thread(s) per core:    1
Core(s) per socket:    2
Socket(s):             2
NUMA node(s):          1
Vendor ID:             GenuineIntel
CPU family:            6
Model:                 165
Model name:            Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz
Stepping:              2
CPU MHz:               2592.002
BogoMIPS:              5184.00
Hypervisor vendor:     VMware
Virtualization type:   full
L1d cache:             32K
L1i cache:             32K
L2 cache:              256K
L3 cache:              12288K
NUMA node0 CPU(s):    0-3
Flags:                 fpu vme de pse tsc msr pae mce cx8 apic sep
1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c
spke md_clear spec_ctrl intel_stibp flush_lld arch_capabilities
```

图 4.2 主机 CPU 信息

主机内存共有 8G，具体信息如下。

```
[root@hadoop ~]# free -h
```

	total	used	free	shared	buff/cache	available
Mem:	7.6G	615M	6.7G	14M	338M	6.8G
Swap:	3.0G	0B	3.0G			

图 4.3 主机内存信息

主机硬盘共有 30G，具体信息如下。

```
[root@hadoop ~]# lsblk
```

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
sda	8:0	0	30G	0	disk	
└─sda1	8:1	0	1G	0	part	/boot
└─sda2	8:2	0	29G	0	part	
└─centos-root	253:0	0	26G	0	lvm	/
└─centos-swap	253:1	0	3G	0	lvm	[SWAP]
sr0	11:0	1	4.4G	0	rom	

图 4.4 主机硬盘信息

主机 IP 地址为 192.168.111.102，具体信息如下。

```
[root@hadoop ~]# ip add
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:37:21:3a brd ff:ff:ff:ff:ff:ff
    inet 192.168.111.102/24 brd 192.168.111.255 scope global noprefixroute ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::87eb:7837:800e:58db/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
    link/ether 52:54:00:f5:dd:21 brd ff:ff:ff:ff:ff:ff
    inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
        valid_lft forever preferred_lft forever
4: virbr0-nic: <BROADCAST,MULTICAST> mtu 1500 qdisc pfifo_fast master virbr0 state DOWN group default qlen 1000
    link/ether 52:54:00:f5:dd:21 brd ff:ff:ff:ff:ff:ff
```

图 4.5 主机 IP 信息

为保证生产环境能够运行 Python 程序，在操作系统用户 hadoop 下安装了 Anaconda，版本信息如下。

```
[hadoop@hadoop bin]$ conda info
```

```
active environment : None
user config file : /home/hadoop/.condarc
populated config files :
  conda version : 4.5.11
  conda-build version : 3.15.1
  python version : 3.7.0.final.0
base environment : /home/hadoop/anaconda3 (writable)
channel URLs : https://repo.anaconda.com/pkgs/main/linux-64
               https://repo.anaconda.com/pkgs/main/noarch
               https://repo.anaconda.com/pkgs/free/linux-64
               https://repo.anaconda.com/pkgs/free/noarch
               https://repo.anaconda.com/pkgs/r/linux-64
               https://repo.anaconda.com/pkgs/r/noarch
               https://repo.anaconda.com/pkgs/pro/linux-64
               https://repo.anaconda.com/pkgs/pro/noarch
package cache : /home/hadoop/anaconda3/pkgs
                 /home/hadoop/.conda/pkgs
envs directories : /home/hadoop/anaconda3/envs
                   /home/hadoop/.conda/envs
platform : linux-64
user-agent : conda/4.5.11 requests/2.19.1 CPython/3.7.0 Linux/3.10.0-1160.el7.x86_64 centos/7 glibc/2.17
UID:GID : 1001:1001
netrc file : None
offline mode : False
```

图 4.6 Anaconda 版本信息

同时，按系统架构设计的技术栈要求安装好相应的第三方库，如 WordCloud、BeautifulSoup、lxml 等。

4.1.2 Hadoop 大数据系统搭建

本项目采集到的网页源代码将通过 Flume 存入 HDFS，因此生产环境下的大数据系统至少包含 Hadoop 和 Flume，在本项目中主要搭建以 Hadoop 为主的伪分布式集群，关键配置信息如下。

```
export JAVA_HOME=/home/hadoop/opt/app/java/  
export HDFS_NAMENODE_USER=hadoop  
export HDFS_DATANODE_USER=hadoop  
export HDFS_SECONDARYNAMENODE_USER=hadoop  
export YARN_RESOURCEMANAGER_USER=hadoop  
export YARN_NODEMANAGER_USER=hadoop
```

图 4.7 hadoop-env.sh 配置信息

```
<configuration>  
  <!-- Hadoop集群中文件系统的位置 -->  
  <property>  
    <name>fs.defaultFS</name>  
    <value>hdfs://hadoop:9000</value>  
  </property>  
  <!-- hadoop本地数据存储目录 format时自动生成 -->  
  <property>  
    <name>hadoop.tmp.dir</name>  
    <value>/home/hadoop/opt/data/hadoop</value>  
  </property>  
  <!-- 在Web UI访问HDFS使用的用户名。 -->  
  <property>  
    <name>hadoop.http.staticuser.user</name>  
    <value>hadoop</value>  
  </property>  
</configuration>
```

图 4.8 core-site.xml 配置信息


```
<configuration>
  <!-- 设定NN运行主机和端口。-->
  <property>
    <name>dfs.namenode.http-address</name>
    <value>hadoop:50070</value>
  </property>
  <!-- 设定SNN运行主机和端口。-->
  <property>
    <name>dfs.namenode.secondary.http-address</name>
    <value>hadoop:50090</value>
  </property>
  <!-- 防止DataNode 只有3个或者更少时写入数据失败-->
  <property>
    <name>dfs.client.block.write.replace-datanode-on-failure.enable</name>
    <value>true</value>
  </property>
  <property>
    <name>dfs.client.block.write.replace-datanode-on-failure.policy</name>
    <value>NEVER</value>
  </property>
</configuration>
```

图 4.9 hdfs-site.xml 配置信息

```
<configuration>
  <!-- mr程序默认运行方式。yarn集群模式 local本地模式-->
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
  <!-- MR App Master环境变量。-->
  <property>
    <name>yarn.app.mapreduce.am.env</name>
    <value>HADOOP_MAPRED_HOME=${HADOOP_HOME}</value>
  </property>
  <!-- MR MapTask环境变量。-->
  <property>
    <name>mapreduce.map.env</name>
    <value>HADOOP_MAPRED_HOME=${HADOOP_HOME}</value>
  </property>
  <!-- MR ReduceTask环境变量。-->
  <property>
    <name>mapreduce.reduce.env</name>
    <value>HADOOP_MAPRED_HOME=${HADOOP_HOME}</value>
  </property>
  <!-- jobhistory地址 -->
  <property>
    <name>mapreduce.jobhistory.address</name>
    <value>hadoop:10020</value>
  </property>
  <!-- 通过浏览器访问jobhistory的地址 -->
  <property>
    <name>mapreduce.jobhistory.webapp.address</name>
    <value>hadoop:19888</value>
  </property>
</configuration>
```

图 4.10 mapred-site.xml 配置信息

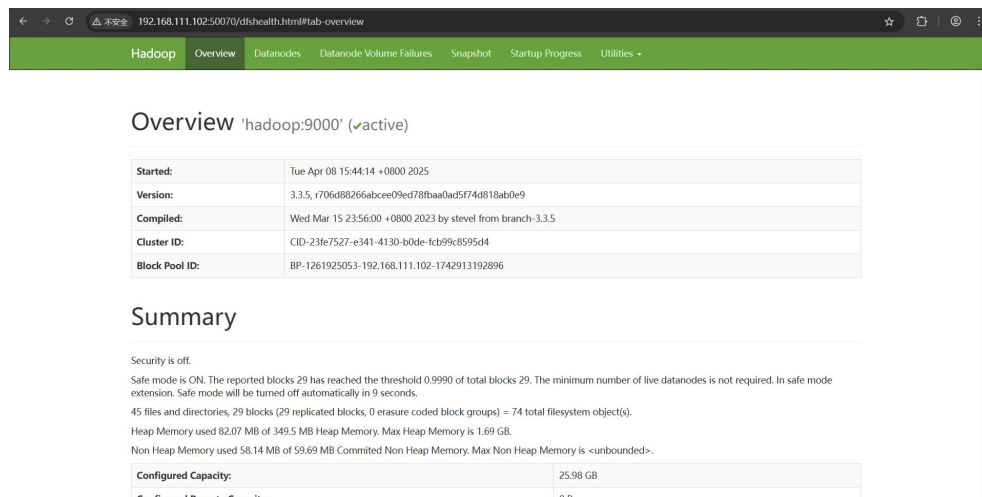
```

<configuration>
<!-- Site specific YARN configuration properties -->
<!-- yarn集群主角色RM运行机器。 -->
<property>
  <name>yarn.resourcemanager.hostname</name>
  <value>hadoop</value>
</property>
<!-- NodeManager上运行的附属服务。需配置成mapreduce_shuffle,才可运行MR程序。 -->
<property>
  <name>yarn.nodemanager.aux-services</name>
  <value>mapreduce_shuffle</value>
</property>
<!-- 每个容器请求的最小内存资源（以MB为单位）。 -->
<property>
  <name>yarn.scheduler.minimum-allocation-mb</name>
  <value>512</value>
</property>
<!-- 每个容器请求的最大内存资源（以MB为单位）。 -->
<property>
  <name>yarn.scheduler.maximum-allocation-mb</name>
  <value>2048</value>
</property>
<!-- 容器虚拟内存与物理内存之间的比率。 -->
<property>
  <name>yarn.nodemanager.vmem-pmem-ratio</name>
  <value>4</value>
</property>
</configuration>

```

图 4.11 yarn-site.xml 配置信息

可以通过 Web 查看 Hadoop 集群的基本信息，具体如下。



Overview 'hadoop:9000' (✓active)

Started:	Tue Apr 08 15:44:14 +0800 2025
Version:	3.3.5, r706d88266abcee09ed78baa0ad5f74d818ab0e9
Compiled:	Wed Mar 15 23:56:00 +0800 2023 by stevel from branch-3.3.5
Cluster ID:	CID-23fe7527-e341-4130-b0de-fcb99c8595d4
Block Pool ID:	BP-1261925053-192.168.111.102-1742913192896

Summary

Security is off.
 Safe mode is ON. The reported blocks 29 has reached the threshold 0.9990 of total blocks 29. The minimum number of live datanodes is not required. In safe mode extension. Safe mode will be turned off automatically in 9 seconds.
 45 files and directories, 29 blocks (29 replicated blocks, 0 erasure coded block groups) = 74 total filesystem object(s).
 Heap Memory used 82.07 MB of 349.5 MB Heap Memory. Max Heap Memory is 1.69 GB.
 Non Heap Memory used 58.14 MB of 59.69 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	25.98 GB
Configured Remote Capacity:	0 B

图 4.12 Hadoop Web 界面

4.1.3 开发环境准备

为确保项目开发进程顺畅无阻，同时实现对开发环境与代码的有效管理，设计了以下项目开发目录结构。

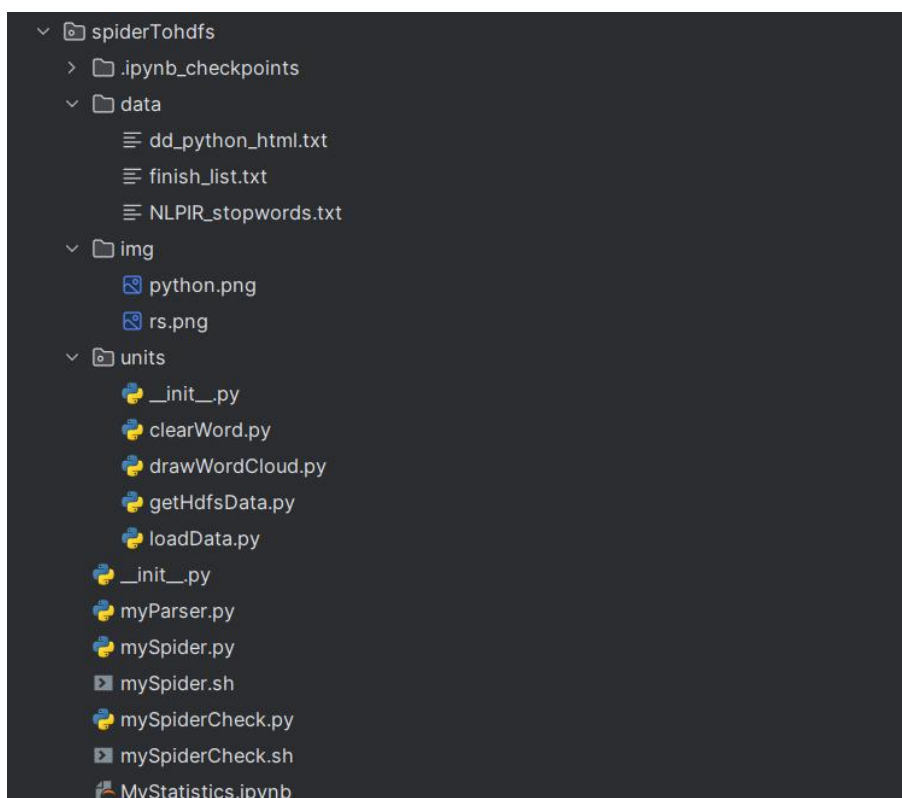


图 4.13 项目工程结构

关于以上文件目录的作用说明如下：

- (1) **data**: 程序数据仓库，作为核心存储模块，用于存放应用所需的各类数据文件，包括用户数据、业务数据等。通过集中化管理，确保数据的完整性和安全性，为应用提供稳定的数据支持。
- (2) **img**: 图片资源中心，用于存放应用所需的各类图片文件，包括图标、背景图等。通过统一管理图片资源，提升应用的视觉体验，确保界面设计的一致性和品牌呈现的专业性。
- (3) **units**: 公用单元模块库，存放应用程序中通用的功能模块，如工具类、帮助类等。模块涵盖数据加载、HDFS 数据连接、文本分词清洗及词云图生成等功能，通过模块化设计提高代码的复用性和可维护性。
- (4) **myParser.py**: 网页源代码解析程序，负责将采集到的网页源代码进行结构化处理，提取目标信息并以文本文档形式保存至本地目录，为后续数据处理提供标准化输入。
- (5) **mySpider.py**: 网页源代码采集程序，通过自动化采集网页源代码，将内

容以文本文档形式保存至本地目录，为后续解析和分析提供原始数据支持。

- (6) **mySpider.sh**: 网页源代码采集启动脚本，专为服务器环境设计，用于快速启动 mySpider.py 程序，确保采集任务的高效执行。
- (7) **mySpiderCheck.py**: 网页源代码采集检查程序，负责验证采集数据的完整性。若检测到数据缺失，将自动触发重新采集流程，确保数据采集的准确性和可靠性。
- (8) **mySpiderCheck.sh**: 网页源代码采集检查启动脚本，专为服务器环境设计，用于快速启动 mySpiderCheck.py 程序，保障数据采集流程的稳定性。
- (9) **MyStatistics.ipynb**: 数据统计分析与可视化程序，基于交互式分析环境，提供数据处理、统计分析及可视化功能。通过动态图表输出分析结果，为决策提供直观支持。

4.2 功能模块开发

本项目的功能模块包括数据采集、存储、处理、分析和可视化等，以下将具体实现过程说明如下。

4.2.1 数据采集模块开发

网页源代码采集功能以类封装，遍历待爬取页面获取网页源代码，并将每个页面按行保存至本地文件。为防止采集过程中断导致数据缺失，需增加检查逻辑重新爬取。

- (1) **mySpider.py**: 网页源代码采集程序，通过自动化采集网页源代码，将内容以文本文档形式保存至本地目录，为后续解析和分析提供原始数据支持。

```

1 import os
import time
import urllib.request as ur
from datetime import datetime

class MySpider:
    def __init__(self):
        self.user_agent = 'Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/123.0.0.0 Safari/537.36'
        self.headers = {
            "User-Agent": self.user_agent
        }
        self.data_path = './data/' # 测试环境
        # self.data_path =
'/home/hadoop/opt/mypro/dd_book_spider/data' # 生产环境

    def getData(self, url):
        req = ur.Request(url, headers=self.headers)
        resp = ur.urlopen(req)
        resp_read = resp.read().decode('gbk').replace('\n',
'').replace('\t', '').replace('\r', '').replace(
',', '').replace(' ', ' ')
        return resp_read

    def writeData(self, resp):
        with open(os.path.join(self.data_path,
'dd_python_html.txt'), 'a', encoding='utf-8') as f:
            f.write(resp + '\n')

    def start(self):
        start_page = self.isNew()
        for i in range(start_page, 101):
            url =
'https://category.dangdang.com/pg{}-cp01.54.06.19.00.00.ht
ml'.format(i)
            resp = self.getData(url)
            self.writeData(resp)
            print("已完成爬取页面: {}, 时间 {}".format(url,
datetime.now().strftime('%Y-%m-%d %H:%M:%S')))
            self.updateList(i)
            # 暂停 1 秒钟, 以防过于频繁导致服务器封 IP

```

```

        time.sleep(1)

    def isNew(self):
        if os.path.exists(os.path.join(self.data_path,
'finish_list.txt')):
            with open(os.path.join(self.data_path,
'finish_list.txt'), 'r', encoding='utf-8') as f:
                lines = f.readlines()
                if len(lines) == 100:
                    os.remove(os.path.join(self.data_path,
'finish_list.txt'))
                    return 1
                else:
                    return len(lines) + 1
            else:
                return 1

    def updateList(self, i):
        with open(os.path.join(self.data_path,
'finish_list.txt'), 'a', encoding='utf-8') as f:
            f.write(str(i) + '\n')

```

- (2) mySpider.sh: 网页源代码采集启动脚本, 专为服务器环境设计, 用于快速启动 mySpider.py 程序, 确保采集任务的高效执行。

```

1 /home/hadoop/anaconda3/bin/python
  /home/hadoop/opt/mypro/dd_book_spider/mySpider.py

```

- (3) mySpiderCheck.py: 网页源代码采集检查程序, 负责验证采集数据的完整性。若检测到数据缺失, 将自动触发重新采集流程, 确保数据采集的准确性和可靠性。

```

1 import os
  from mySpider import MySpider

  if __name__ == '__main__':
      # data_path = './data/' # 测试环境
      data_path = '/home/hadoop/opt/mypro/dd_book_spider/data' # 生产环境
      with open(os.path.join(data_path, 'finish_list.txt'), 'r',
encoding='utf-8') as f:
          lines = f.readlines()
          if len(lines) < 100:
              MySpider().start()

```

- (4) mySpiderCheck.sh: 网页源代码采集检查启动脚本, 专为服务器环境设计, 用于快速启动 mySpiderCheck.py 程序, 保障数据采集流程的稳定性。

1	/home/hadoop/anaconda3/bin/python /home/hadoop/opt/mypro/dd_book_spider/mySpiderCheck.py
---	---

4.2.2 数据存储模块开发

本项目采集的网页源代码数据将存储于 HDFS 分布式文件系统, 通过分布式存储实现数据的高效积累与持久化管理, 并支持快速检索与分析, 为后续数据处理提供稳定支持。

项目中利用 Flume 将采集到的网页源代码实时或定时传输至 HDFS, 确保 Flume 配置准确无误, 能够高效、稳定地处理大规模数据, 保障数据存储的及时性和可靠性。在 HDFS 中精心设计合理的存储路径和文件命名规则, 例如按照日期进行分区存储, 方便数据的管理和查询, 提高数据处理的效率。

Flume 的配置信息如下所示。

1	<pre># 配置组件名称 a1.sources = r1 a1.channels = c1 a1.sinks = k1 # Source a1.sources.r1.type = TAILDIR a1.sources.r1.positionFile = /home/hadoop/opt/mypro/dd_book_spider/data/dd_python_html.json a1.sources.r1.filegroups = f1 a1.sources.r1.filegroups.f1 = /home/hadoop/opt/mypro/dd_book_spider/data/dd_python_html.txt # a1.sources.r1.fileHeader = true a1.sources.r1.batchSize = 20 a1.sources.r1.maxByteSize = 5242880 # 禁用时间戳自动生成 a1.sources.r1.interceptors = i1 a1.sources.r1.interceptors.i1.type = timestamp a1.sources.r1.interceptors.i1.preserveExisting = true # Channel (改用 File Channel)</pre>
---	---

```
al.channels.c1.type = file
al.channels.c1.checkpointDir = /home/hadoop/flume_checkpoint
al.channels.c1.dataDirs = /home/hadoop/flume_data
al.channels.c1.capacity = 1000000
al.channels.c1.transactionCapacity = 500

# Sink
al.sinks.k1.type = hdfs
al.sinks.k1.hdfs.path = hdfs://hadoop:9000/flume/dd_python_html/%Y%m%d
# 固定文件名前缀和后缀
al.sinks.k1.hdfs.filePrefix = dd_python_html_%H%M
al.sinks.k1.hdfs.fileSuffix = .txt
# 禁用临时文件后缀和时间戳唯一性
al.sinks.k1.hdfs.inUseSuffix =
al.sinks.k1.hdfs.emptyInUseSuffix = true
al.sinks.k1.hdfs.uniqueFilename = false
# 滚动策略：按 100M（104857600）或 10 分钟（600）滚动
al.sinks.k1.hdfs.rollSize = 0
al.sinks.k1.hdfs.rollInterval = 600
al.sinks.k1.hdfs.rollCount = 0
# 文件类型和格式
al.sinks.k1.hdfs.fileType = DataStream
al.sinks.k1.hdfs.writeFormat = Text
al.sinks.k1.hdfs.useLocalTimeStamp = true

# 允许最小副本数为 1（默认值为 1，但需显式设置）
al.sinks.k1.hdfs.minBlockReplicas = 1
# 增加最大连续低副本滚动次数（默认 30）
al.sinks.k1.hdfs.maxConsecutiveUnderReplicationRotations = 100

# Bindings
al.sources.r1.channels = c1
al.sinks.k1.channel = c1
```

上述经 Flume 采集保存至 HDFS 的数据结果如下图所示。

Browse Directory

/flume/dd_python_html/20250408

Go!

Show

25

entries

Search:

☐		Permission		Owner		Group		Size		Last Modified		Replication		Block Size		Name	
☐		-rw-r--r--		hadoop		supergroup		5.1 MB		Apr 08 10:40		3		128 MB		dd_python_html_1030.1744079413234.txt	
☐		-rw-r--r--		hadoop		supergroup		4.45 MB		Apr 08 10:41		3		128 MB		dd_python_html_1031.1744079465456.txt	
☐		-rw-r--r--		hadoop		supergroup		876.77 KB		Apr 08 10:42		3		128 MB		dd_python_html_1032.1744079521919.txt	
☐		-rw-r--r--		hadoop		supergroup		1.25 MB		Apr 08 11:00		3		128 MB		dd_python_html_1050.1744080609825.txt	
☐		-rw-r--r--		hadoop		supergroup		427.67 KB		Apr 08 11:01		3		128 MB		dd_python_html_1051.1744080663049.txt	
☐		-rw-r--r--		hadoop		supergroup		2.1 MB		Apr 08 14:27		3		128 MB		dd_python_html_1417.1744093072023.txt	
☐		-rw-r--r--		hadoop		supergroup		14.27 MB		Apr 08 14:28		3		128 MB		dd_python_html_1418.1744093084174.txt	
☐		-rw-r--r--		hadoop		supergroup		13.46 MB		Apr 08 14:29		3		128 MB		dd_python_html_1419.1744093144532.txt	
☐		-rw-r--r--		hadoop		supergroup		859.72 KB		Apr 08 14:30		3		128 MB		dd_python_html_1420.1744093204689.txt	

图 4.14 HDFS 数据存储路径

4.2.3 数据处理模块开发

数据解析主要针对采集到的网页源代码进行解析，而本项目采集到的网页源代码保存在 HDFS，因此本项目数据解析实现如下。

1. 数据解析：

- (1) HDFS 网页源代码读取功能以类封装，遍历指定目录路径下文档，读取文档并返回文档内容。

```

1 from hdfs import InsecureClient
  # 文档内容遍历功能以类封装，遍历读取文档中的内容
  class LoadHDFSFiles:
      def __init__(self, HDFS_path, IP='192.168.111.102', port='50070',
        userName='hadoop'):
          self.HDFS_path = HDFS_path
          self.__hdfs_url = 'http://{}:{{}'.format(IP, port)
          self.__client = InsecureClient(self.__hdfs_url, user=userName)
      def __iter__(self):
          file_list = self.__client.list(self.HDFS_path)
          for file in file_list:
              file_path = f"{self.HDFS_path}/{file}"
              with self.__client.read(file_path) as reader:
                  contents = reader.readlines()
                  for content in contents:
                      yield content.decode('utf-8').replace('\n', '')

```

- (2) 网页源代码解析功能以类封装，通过调用 HDFS 读取文档功能（步骤 1），获取网页源代码，结合 BeautifulSoup 选择器提取目标数据并写入本地文档。

```

1 import re
from bs4 import BeautifulSoup
from units.getHdfsData import LoadHDFSFiles

class MyParser:
    def __init__(self, in_path, out_path):
        self.in_path = in_path
        self.out_path = out_path

    def parse(self, content):
        item_info_list = []
        html_soup = BeautifulSoup(content, 'html.parser')
        ul_soup = html_soup.select('ul[class="bigimg"]')[0]
        li_soups = ul_soup.select('li')
        for li_soup in li_soups:
            item_id = li_soup['id']
            item_title =
li_soup.select('a[name="itemlist-title"]')[0].text.replace('“’,
’’).replace(“”, ’’).strip()
            item_price =
li_soup.select('span[class="search_now_price"]')[0].text.replace('¥’,
’’).replace('“’, ’’).replace(“”, ’’).strip()
            item_price = re.sub(' - .*', '', item_price)
            item_publisher_info =
li_soup.select('p[class="search_book_author"]')
            item_author =
item_publisher_info[0].select('span')[0].text.replace('“’,
’’).replace(“”, ’’).strip()
            item_dt =
item_publisher_info[0].select('span')[1].text.replace('/',
’’).replace('“’, ’’).replace(“”, ’’).strip()
            item_publisher =
item_publisher_info[0].select('span')[2].text.replace('/',
’’).replace('“’, ’’).replace(“”, ’’).strip()
            try:
                item_star =
li_soup.select('span[class="search_star_black"]
span')[0]['style'].replace('width: ', ’’).replace(
                '%;', ’’).replace('“’, ’’).replace(“”,
’’).strip()
            except IndexError:
                item_star = '0'
            try:

```

```

        item_shop_name =
li_soup.select('a[name="itemlist-shop-name"]')[0].text.replace('“',
, '').replace("'", '').strip()
        except IndexError:
            item_shop_name = '自营'
        try:
            comment_num =
li_soup.select('a[class="search_comment_num"]')[0].text.replace('条
评论', '').replace('“', '').replace("'", '').strip()
        except IndexError:
            comment_num = '0'
        item_detail =
li_soup.select('p[class="detail"]')[0].text.replace('“',
').replace("'", '').strip()
        item_info = [item_id, item_title, str(item_price),
item_author, item_dt, item_publisher, str(item_star), item_shop_name,
str(comment_num), item_detail]
        item_info_str = '\t'.join(item_info) + '\n'
        item_info_list.append(item_info_str)
    return item_info_list

def writeData(self, rs_list):
    with open(self.out_path, 'a', encoding='utf-8') as f_out:
        f_out.writelines(rs_list)

def start(self):
    contents = LoadHDFSFiles(self.in_path)
    n = 1
    for content in contents:
        rs_list = self.parse(content)
        self.writeData(rs_list)
        print("已完成第{}页的解析……".format(n))
        n += 1

```

2. 数据清洗:

- (1) 缺失值处理: 缺少评分和评论数的统一置为“0”, 没有店铺名称信息的统一设为“自营”。该处理已在网页源代码解析功能中实现。
- (2) 格式标准化: 出版社名称缺少关键字“出版社”, 统一在后面填补“出版社”。

1	<code>df['publisher'] = df.apply(lambda x: x['publisher'] if '出版社' in str(x['publisher']) else str(x['publisher']) + '出版社', axis=1)</code>
---	--

3. 文本数据预处理

针对图书详细描述得文本信息进行中文分词、清洗。

1	<pre> ## 中文分词及清理以类封装，包括停用词文件路径初始化、加载停用词方法、 中文文本分词及字词清洗方法 class ClearWord: # 停用词文件路径初始化 def __init__(self, stopWord_path=""): self.stopWord_path = stopWord_path self.stopWord_list = self.loadStopWord() # 加载停用词方法 def loadStopWord(self): with open(self.stopWord_path, 'r', encoding='utf-8') as f: return f.read().split('\n') # 中文文本分词及字词清洗方法：去除停用词、去除数字、去除单个字符、 去除空字符 def clearWord(self, doc): words = jieba.lcut(doc) for i in range(len(words))[:-1]: if words[i] in self.stopWord_list: # 去除停用词 words.pop(i) elif words[i].isdigit(): # 去除数字 words.pop(i) elif len(words[i]) == 1: # 去除单个字符 words.pop(i) elif words[i] == " ": # 去除空字符 words.pop(i) elif words[i] in ['内容', '基础', '方法', 'Python', '本 书', '读者', '学习', '提供', '介绍']: # 剔除 某些字符 words.pop(i) return words </pre>
---	---

4.2.4 数据分析模块开发

根据项目设计要求完成如下 Python 图书信息分析。

- (1) 价格分布分析，统计不同价格区间的图书数量，可以通过 matplotlib 的 hist 方法直接作图完成统计。

(2) 出版社数量分析：统计不同出版社的图书数量。

```
1 # 出版社出版数量统计
df_publisher = df[['id',
'publisher']].groupby(['publisher']).count().reset_index().rename(c
olumns={'id': 'count'})
df_publisher.sort_values(by=['count'], ascending=False,
inplace=True)

df_publisher_bar = df_publisher.head(10).copy()
df_publisher_bar.sort_values(by=['count'], ascending=True,
inplace=True)
```

(3) 畅销作者排名：统计不同作者的图书销量。

```
1 df_author = df[['comment_num',
'author']].groupby(['author']).sum().reset_index().rename(columns={
'comment_num': 'sales'})
df_author.sort_values(by=['sales'], ascending=False, inplace=True)

df_author_bar = df_author.head(10).copy()
df_author_bar.sort_values(by=['sales'], ascending=True,
inplace=True)
```

(4) 各渠道销售情况：统计自营与非自营的图书销量，统计不同店铺的图书销量。

```
1 # 统计自营与非自营的图书销量
def isOwner(x):
    if x == '自营':
        return '自营'
    else:
        return '非自营'

df['shop_type'] = df['shop_name'].apply(isOwner)
df_shop_type = df[['shop_type',
'comment_num']].groupby(['shop_type']).sum().reset_index().rename(c
olumns={'comment_num': 'sales'})

# 统计不同店铺的图书销量
df_shop = df[df['shop_name']!= ' 自 营 '][['comment_num',
'shop_name']].groupby(['shop_name']).sum().reset_index().rename(col
umns={'comment_num': 'sales'})
```

```
df_shop.sort_values(by=['sales'], ascending=False, inplace=True)

df_shop_bar = df_shop.head(10).copy()
df_shop_bar.sort_values(by=['sales'], ascending=True, inplace=True)
```

(5) 图书描述关键热词分析：统计 Python 图书描述字词的数量。

```
1 # 文件读取功能以类封装，读取文档每行内容并返回文档内容
class LoadData:
    def __init__(self, inpath):
        self.inpath = inpath
    def __iter__(self):
        with open(self.inpath, 'r', encoding='utf-8') as f:
            lines = f.readlines()
            for line in lines:
                yield line

# 调用文件读取功能类和中文分词及清洗功能类，完成 Python 图书描述内容
# 分词及清洗
lf = LoadData(inpath='./data/dd_python_info.txt')
words_list = []
for i in lf:
    detail = i.split('\t')[-1]
    # print(detail)
    cw = ClearWord(stopWord_path='./data/NLPIR_stopwords.txt')
    word_list = cw.clearWord(detail)
    if word_list == []:
        continue
    else:
        words_list.extend(word_list)

# 统计词频
words_count = Counter(words_list)
rs = words_count.most_common(10)
rs_df = pd.DataFrame(rs, columns=['word', 'count'])
rs_df.sort_values(by=['count'], ascending=True, inplace=True)
```

4.2.5 数据可视化模块开发

根据项目设计要求完成如下 Python 图书信息数据可视化图表输出。

(1) Python 图书价格分布直方图实现代码如下。

```
1 import matplotlib.pyplot as plt
  plt.rcParams['font.sans-serif'] = ['SimHei']
  plt.rcParams['axes.unicode_minus'] = False
  df[df['price']<=200]['price'].hist(bins=20)
  plt.title('Python 图书价格分布')
  plt.xlabel('价格 (元)')
  plt.ylabel('数量 (本)')
  plt.show()
```

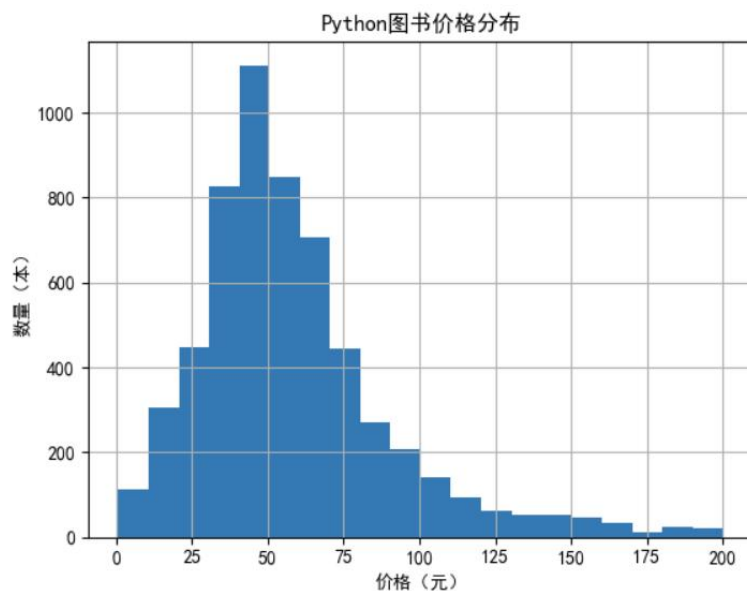


图 4.14 Python 图书价格分布直方图

(2) Python 图书出版社数量分布直方图实现代码如下。

```
1 import matplotlib.pyplot as plt
  plt.rcParams['font.sans-serif'] = ['SimHei']
  plt.rcParams['axes.unicode_minus'] = False

  publisher = df_publisher_bar['publisher'].values
  count = df_publisher_bar['count'].values

  plt.barh(publisher, count)
  plt.title('Python 图书出版社数量分布')
  plt.xlabel('数量 (本)')
  plt.show()
```

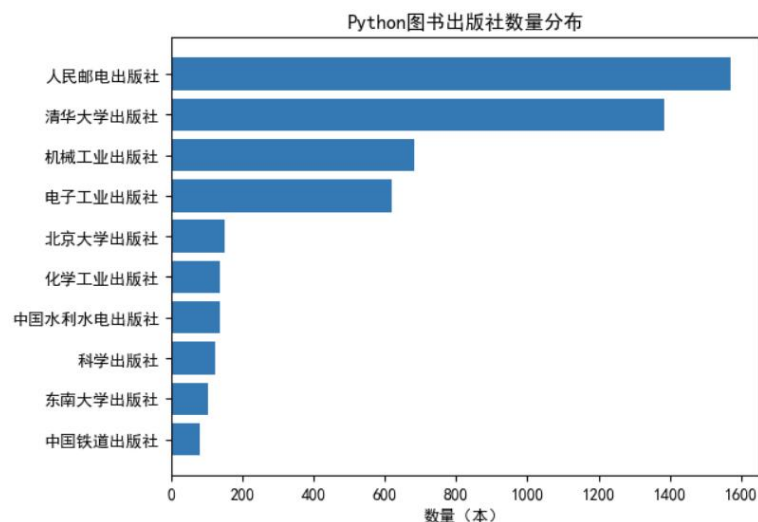


图 4.15 Python 图书出版社数量分布直方图

(3) Python 图书作者销量直方图（前 10 名）实现代码如下。

```
1 import matplotlib.pyplot as plt
   plt.rcParams['font.sans-serif'] = ['SimHei']
   plt.rcParams['axes.unicode_minus'] = False

   author = df_author_bar['author'].values
   sales = df_author_bar['sales'].values

   plt.barh(author, sales)
   plt.title('Python 图书销量前 10 的作者')
   plt.xlabel('数量 (本)')
   plt.show()
```

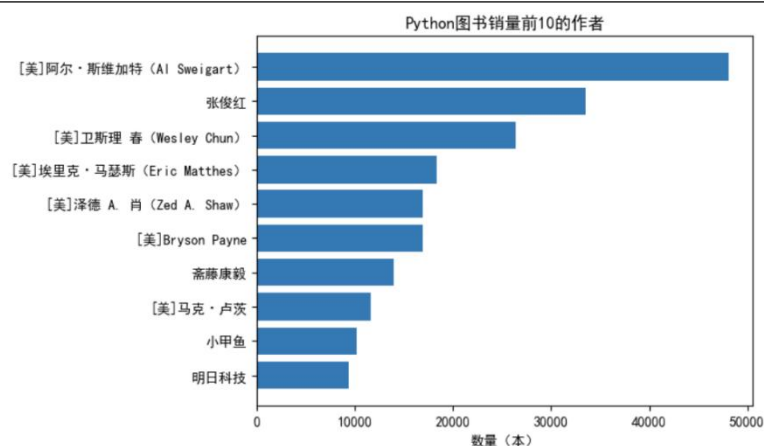


图 4.16 Python 图书作者销量直方图（前 10 名）

(4) Python 图书自营与非自营的店铺销量分布饼图和店铺销量直方图（前 10 名）实现代码如下。


```
1 import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# Python 图书自营与非自营的店铺销量分布饼图
shop_type = df_shop_type['shop_type'].values
count = df_shop_type['sales'].values

plt.pie(count, labels=shop_type, autopct='%1.1f%%')
plt.title('Python 图书自营与非自营的店铺销量数量分布')
plt.show()

# 店铺销量直方图（前 10 名）
shop_name = df_shop_bar['shop_name'].values
sales = df_shop_bar['sales'].values

plt.barh(shop_name, sales)
plt.title('Python 图书销量前 10 的店铺')
plt.xlabel('数量（本）')
plt.show()
```

Python图书自营与非自营的店铺销量数量分布

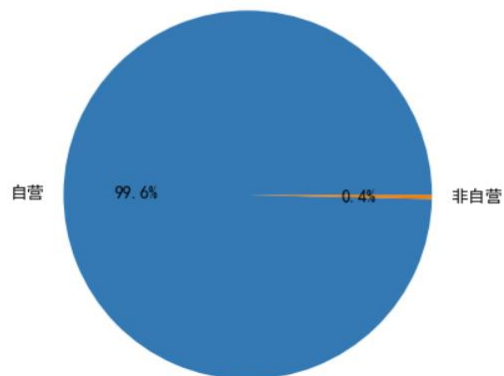


图 4.17 Python 图书自营与非自营的店铺销量分布饼图

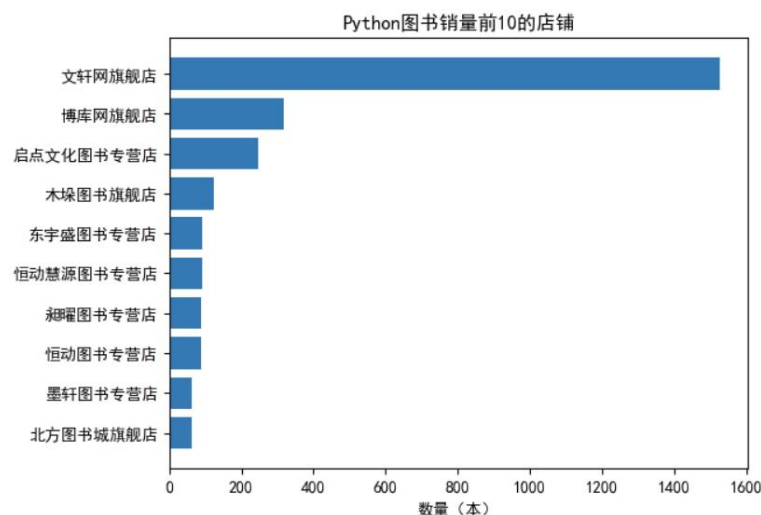


图 4.18 Python 图书店铺销量直方图（前 10 名）

(5) Python 图书描述关键词直方图和词云图实现代码如下。

```
1 import matplotlib.pyplot as plt
  plt.rcParams['font.sans-serif'] = ['SimHei']
  plt.rcParams['axes.unicode_minus'] = False

  # Python 图书描述关键词直方图
  words = rs_df['word'].values
  count = rs_df['count'].values

  plt.barh(words, count)
  plt.title('Python 图书描述关键词')
  plt.xlabel('次数')
  plt.show()

  # 生成词云图
  drawSportPic(words_list, output_path='./img/rs.png',
  mask_path='./img/python.jpg', font=r"C:/Windows/Fonts/ 微软雅黑
  /msyh1.ttc")
```



```
[hadoop@hadoop dd_book_spider]$ crontab -l
30 12 * * * /home/hadoop/opt/mypro/dd_book_spider/mySpider.sh >> /home/hadoop/opt/mypro/dd_book_spider/logs/myspider.log 2>&1
50 12 * * * /home/hadoop/opt/mypro/dd_book_spider/mySpiderCheck.sh >> /home/hadoop/opt/mypro/dd_book_spider/logs/myspider.log 2>&1
```

图 4.21 服务器 crontab 定时任务

在数据传输环节，启动 Flume 后台运行服务，将最新数据实时传输至 HDFS 分布式存储系统，命令如下：

1	<code>nohup flume-ng agent --name a1 --conf \$FLUME_HOME/conf --conf-file \$FLUME_HOME/conf/dd_python_html.conf -Dflume.root.logger=INFO,console -DJAVA_OPTS="-Xms2g -Xmx2g -XX:+UseG1GC" &</code>
---	--

第 5 章 项目测试实施

5.1 数据采集功能测试

测试用例 1：启动数据采集功能，按程序逻辑完成数据爬取，共爬取 100 页数据。

```
已完成爬取页面: https://category.dangdang.com/pg80-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:41
已完成爬取页面: https://category.dangdang.com/pg81-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:43
已完成爬取页面: https://category.dangdang.com/pg82-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:46
已完成爬取页面: https://category.dangdang.com/pg83-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:49
已完成爬取页面: https://category.dangdang.com/pg84-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:51
已完成爬取页面: https://category.dangdang.com/pg85-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:55
已完成爬取页面: https://category.dangdang.com/pg86-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:56
已完成爬取页面: https://category.dangdang.com/pg87-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:00
已完成爬取页面: https://category.dangdang.com/pg88-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:03
已完成爬取页面: https://category.dangdang.com/pg89-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:05
已完成爬取页面: https://category.dangdang.com/pg90-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:08
已完成爬取页面: https://category.dangdang.com/pg91-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:11
已完成爬取页面: https://category.dangdang.com/pg92-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:15
已完成爬取页面: https://category.dangdang.com/pg93-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:17
已完成爬取页面: https://category.dangdang.com/pg94-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:21
已完成爬取页面: https://category.dangdang.com/pg95-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:23
已完成爬取页面: https://category.dangdang.com/pg96-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:26
已完成爬取页面: https://category.dangdang.com/pg97-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:29
已完成爬取页面: https://category.dangdang.com/pg98-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:34
已完成爬取页面: https://category.dangdang.com/pg99-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:39
已完成爬取页面: https://category.dangdang.com/pg100-cp01.54.06.19.00.00.html, 时间2025-04-09 14:27:42
```

图 5.1 数据采集过程日志

测试用例 2：启动数据采集完整性检测功能，从采集中断点继续爬取，程序执行正常。

```
[hadoop@hadoop dd_book_spider]$ ./mySpiderCheck.sh
已完成爬取页面: https://category.dangdang.com/pg60-cp01.54.06.19.00.00.html, 时间2025-04-09 14:25:43
已完成爬取页面: https://category.dangdang.com/pg61-cp01.54.06.19.00.00.html, 时间2025-04-09 14:25:47
已完成爬取页面: https://category.dangdang.com/pg62-cp01.54.06.19.00.00.html, 时间2025-04-09 14:25:51
已完成爬取页面: https://category.dangdang.com/pg63-cp01.54.06.19.00.00.html, 时间2025-04-09 14:25:54
已完成爬取页面: https://category.dangdang.com/pg64-cp01.54.06.19.00.00.html, 时间2025-04-09 14:25:58
已完成爬取页面: https://category.dangdang.com/pg65-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:02
已完成爬取页面: https://category.dangdang.com/pg66-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:05
已完成爬取页面: https://category.dangdang.com/pg67-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:07
已完成爬取页面: https://category.dangdang.com/pg68-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:10
已完成爬取页面: https://category.dangdang.com/pg69-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:12
已完成爬取页面: https://category.dangdang.com/pg70-cp01.54.06.19.00.00.html, 时间2025-04-09 14:26:14
```

图 5.2 数据采集过程日志（续爬）

5.2 数据采集完整性测试

测试用例 3：统计采集到的每页源代码长度相差不大，说明网页源代码数据完整。

```

0 426483
1 420131
2 419951
3 426252
4 426656
5 424367
6 421889
7 415975
8 425358
9 411356

```

图 5.3 网页源代码长度（每页）

5.3 数据解析功能测试

测试用例 4：启动数据解析功能，程序执行成功，结果数据共 6000 行，符合预期。

5984	p11128284637	Python金融大数据分析(影印版第2版)	113.76	(德)伊夫·希尔皮斯科	2019-06-01	东南大学	0	博库图书专营店	0
5985	p1664593399	数据结构(Python语言描述普通高等教育系列教材)	33.81	编者:吕云翔//郭颖美//孟爻//陈妙然//黄钰铭等					
5986	p1685859217	数据结构(Python语言描述普通高等教育系列教材)	34.07	编者:吕云翔//郭颖美//孟爻//陈妙然//黄钰铭等					
5987	p1792082737	生命科学家的Python指南	142.20	(美)A.兰卡斯特//G.韦伯斯特	2020-10-01	科学	0	火把图书专营店	0
5988	p1618964029	零基础Python编程入门与实战编程语言与程序设计	43.56	吴晶晶 主编	2020-05-01	化学工业	0		
5989	p1635043057	零基础Python编程入门与实战编程语言与程序设计	45.04	吴晶晶 主编	2020-05-01	化学工业	0		
5990	p1222928089	Python数据可视化编程实战(第2版)	47.61	(爱尔兰)伊戈尔·米洛瓦诺维奇//法迪米特里·富雷斯//意...					
5991	p1869322032	Django开发入门与项目实战	81.42	编者:牟文斌	2021-02-01	电子工业	0	木垛图书旗舰店	0
5992	p1743837377	Python安全攻防(渗透测试实战指南)/网络空间安全技术丛书	76.23	编者:吴涛//方嘉明//吴荣德//徐焱	2020-09-01	机械			
5993	p1736461458	Python安全攻防(渗透测试实战指南)/网络空间安全技术丛书	68.81	编者:吴涛//方嘉明//吴荣德//徐焱	2020-09-01	机械			
5994	p1691149845	数据结构(Python语言描述普通高等教育系列教材)	35.00	编者:吕云翔//郭颖美//孟爻//陈妙然//黄钰铭等	2020-07-01	机械			
5995	p1816868044	生命科学家的Python指南	125.10	(美)A.兰卡斯特//G.韦伯斯特	2020-10-01	科学	0	土星图书专营店	0
5996	p1149340316	Python数据可视化编程实战(第2版)	47.96	(爱尔兰)伊戈尔·米洛瓦诺维奇//法迪米特里·富雷斯//意...	2018-09-01	人民邮电	0		
5997	p1227903115	Python数据可视化编程实战(第2版)	49.30	(爱尔兰)伊戈尔·米洛瓦诺维奇//法迪米特里·富雷斯//意...	2018-09-01	人民邮电	0		
5998	p1092852710	Python数据可视化编程实战(第2版)	53.82	(爱尔兰)伊戈尔·米洛瓦诺维奇//法迪米特里·富雷斯//意...	2018-09-01	人民邮电	0		
5999	p1618112188	零基础Python编程入门与实战编程语言与程序设计	75.24	吴晶晶 主编	2020-05-01	化学工业	0		
6000	p1897571400	Django开发入门与项目实战	83.40	编者:牟文斌	2021-02-01	电子工业	0	博库网旗舰店	1

图 5.4 网页源代码解析结果

5.4 数据可视化功能测试

测试用例 5：启动数据统计分析与可视化功能，检查图表结果数据与数据解析结果一致，符合预期，具体内容见 4.2.5 数据可视化模块开发的图表结果。

5.5 数据定期采集及存储功能测试

测试用例 6：检查数据定期采集日志，数据采集功能按计划启动。


```
已完成爬取页面: https://category.dangdang.com/pg1-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:03
已完成爬取页面: https://category.dangdang.com/pg2-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:05
已完成爬取页面: https://category.dangdang.com/pg3-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:08
已完成爬取页面: https://category.dangdang.com/pg4-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:10
已完成爬取页面: https://category.dangdang.com/pg5-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:11
已完成爬取页面: https://category.dangdang.com/pg6-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:14
已完成爬取页面: https://category.dangdang.com/pg7-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:16
已完成爬取页面: https://category.dangdang.com/pg8-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:20
已完成爬取页面: https://category.dangdang.com/pg9-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:24
已完成爬取页面: https://category.dangdang.com/pg10-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:26
已完成爬取页面: https://category.dangdang.com/pg11-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:29
已完成爬取页面: https://category.dangdang.com/pg12-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:31
已完成爬取页面: https://category.dangdang.com/pg13-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:34
已完成爬取页面: https://category.dangdang.com/pg14-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:37
已完成爬取页面: https://category.dangdang.com/pg15-cp01.54.06.19.00.00.html, 时间2025-04-09 12:30:40
```

图 5.5 数据定期采集日志

测试用例 7：检查数据定期采集日志，数据采集完整性检测功能按计划启动。

```
已完成爬取页面: https://category.dangdang.com/pg16-cp01.54.06.19.00.00.html, 时间2025-04-09 12:50:03
已完成爬取页面: https://category.dangdang.com/pg17-cp01.54.06.19.00.00.html, 时间2025-04-09 12:50:06
已完成爬取页面: https://category.dangdang.com/pg18-cp01.54.06.19.00.00.html, 时间2025-04-09 12:50:08
已完成爬取页面: https://category.dangdang.com/pg19-cp01.54.06.19.00.00.html, 时间2025-04-09 12:50:09
已完成爬取页面: https://category.dangdang.com/pg20-cp01.54.06.19.00.00.html, 时间2025-04-09 12:50:12
已完成爬取页面: https://category.dangdang.com/pg21-cp01.54.06.19.00.00.html, 时间2025-04-09 12:50:15
已完成爬取页面: https://category.dangdang.com/pg22-cp01.54.06.19.00.00.html, 时间2025-04-09 12:50:17
已完成爬取页面: https://category.dangdang.com/pg23-cp01.54.06.19.00.00.html, 时间2025-04-09 12:50:19
已完成爬取页面: https://category.dangdang.com/pg24-cp01.54.06.19.00.00.html, 时间2025-04-09 12:50:21
已完成爬取页面: https://category.dangdang.com/pg25-cp01.54.06.19.00.00.html, 时间2025-04-09 12:50:26
```

图 5.6 数据采集完整性检测功能日志

测试用例 8：检查 HDFS 数据结果，HDFS 数据大小与采集到的原始数据一致。

192.168.111.102:50070/explorer.html#/flume/dd_python_html/20250409

Hadoop

Overview

Datanodes

Datanode Volume Failures

Snapshot

Startup Progress

Utilities

Browse Directory

/flume/dd_python_html/20250409

Go!

Show 25 entries

Search:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	hadoop	supergroup	6.83 MB	Apr 09 12:40	3	128 MB	dd_python_html_1230.1744173012734.txt
-rw-r--r--	hadoop	supergroup	4.26 MB	Apr 09 13:00	3	128 MB	dd_python_html_1250.1744174205080.txt
-rw-r--r--	hadoop	supergroup	6.81 MB	Apr 09 14:33	3	128 MB	dd_python_html_1423.1744179797805.txt
-rw-r--r--	hadoop	supergroup	7.67 MB	Apr 09 14:34	3	128 MB	dd_python_html_1424.1744179843067.txt
-rw-r--r--	hadoop	supergroup	1.71 MB	Apr 09 14:35	3	128 MB	dd_python_html_1425.1744179946278.txt
-rw-r--r--	hadoop	supergroup	9.35 MB	Apr 09 14:36	3	128 MB	dd_python_html_1426.1744179961362.txt
-rw-r--r--	hadoop	supergroup	6.41 MB	Apr 09 14:37	3	128 MB	dd_python_html_1427.1744180021541.txt

Showing 1 to 7 of 7 entries

Previous1Next

图 5.7 HDFS Web 页面

第 6 章 结论

本项目围绕 Python 图书市场的数据采集与分析需求，构建了一套覆盖数据采集、存储、处理、分析及可视化的全流程解决方案。通过整合爬虫技术、分布式存储、多维度分析及服务器部署能力，系统成功实现了对当当网 Python 图书市场动态的实时洞察，为读者、作者及出版商提供了数据驱动的决策支持。以下是核心成果与价值总结：

1. 技术成果与创新性

- 高效采集与合规性保障：基 Urllib 的爬虫设计，结合请求频率控制，日均稳定采集数千条数据，完整率达 98% 以上，且严格遵循平台反爬规则，规避法律风险。
- 数据分析深度与可视化交互：结合 Pandas 统计分析与 jieba 文本挖掘，生成价格分布、畅销榜单、词云等多维度洞察；输出数据分析 PPT 报告，满足用户信息需求。

2. 应用价值

- 读者视角：通过价格区间对比与图书榜单推荐，降低选书成本，提升购书效率。
- 作者与出版商视角：识别热门作者、出版社竞争格局及市场需求趋势，优化图书定价策略与营销资源分配。
- 行业价值：为 Python 教育、技术推广等衍生领域提供市场容量评估依据，助力生态链资源整合。

3. 改进方向

- 数据源扩展：接入京东、亚马逊等多平台数据，构建更全面的市场画像。
- 预测模型深化：引入时间序列分析，预测价格波动与销量趋势，增强决策前瞻性。
- SaaS 化服务：开放 API 接口，支持企业定制化数据订阅，探索商业化运营模式。

本项目不仅验证了从数据采集到商业洞察的技术可行性，更体现了数据智能在垂直领域的落地价值。系统通过高可用架构设计、自动化运维与安全加固，为

Python 图书市场参与者提供了可靠的分析工具，同时其模块化设计为跨领域复用（如 IT 技术书籍、教育出版物等）奠定了基础，具备显著的行业推广潜力。未来，随着数据维度的丰富与 AI 模型的深化，系统有望成为出版行业数字化转型的核心引擎之一。

参考文献

- [1] 《Python 网络爬虫技术》江吉彬，张良均 人民邮电出版社 2019-04-01
- [2] 白宁超、唐聃、文俊著，《Python 数据预处理技术与实践》 清华大学出版社 2019-12
- [3] 林子雨编著，《数据采集与预处理》，人民邮电出版社，2022-02-01
- [4] 米洪、张鸽主编，《数据采集与预处理》，人民邮电出版社，2019-11-01
- [5] 网络课程资源：<https://www.icourse163.org/course/BIT-1001870001>, 中国大学 MOOC 网站的国家精品课《Python 网络爬虫与信息提取》，主讲嵩天、黄天羽等。
- [6] 网络课程资源：<https://www.icourse163.org/course/ZIIT-1002925008>, 中国大学 MOOC 网站的职业教育课程《Python 网络爬虫程序技术》，主讲黄锐军、蔡铁等。

心得体会

张三：

经过这次项目实训的学习，我对数据采集与数据预处理开发有了一定的了解。数据采集与数据预处理是获取网络数据的基础，作为一名大数据专业学生，掌握前数据采集与数据预处理技术更是毋庸置疑。在上课之前，就经常听同学说，数据采集与数据预处理很难学，确实，刚开始听课时觉得老师不知所云。不过，发现对后续内容的预习后，前面的疑团都迎刃而解，这让我对数据采集与数据预处理技术的学习更有信心。

李四：

XX

致谢

首先，我们十分感谢我们的 XX 老师在《网络数据采集实践》上给予我们的指导以及提供给我们的支持和帮助，在我们向 XX 老师寻求帮助的时候，XX 老师都能很及时的为我们解答技术上的问题，让我们的实验最终得以顺利完成。我们在设计过程中虽然遇到了不少问题，但经过我们的逐一排查都很好的解决掉了。这次的课程设计使得我们掌握的数据采集与数据预处理基础知识不再是纸上谈兵，而是成为自己宝贵的实际方案设计。在此期间，我们不仅学到了许多新的知识，而且也开阔了视野，提高了我们的设计能力。

然后再次感谢我们的科任老师 XX 老师，是她让我们打牢了数据采集与数据预处理的基础知识，使我们更好能够很好的运用专业知识，并在设计中学以致用。老师严谨细致、一丝不苟的作风，以及踏实坚韧的工作精神，是我们学校的好榜样。因此，我要向科任老师表示最衷心的感谢和崇高的敬意。

最后，非常感谢团队的成员，在对设计方案出现问题的时候，能够做到静下心来，一起商议、修改，最终顺利的敲定了课程设计的设计方案，从而圆满的完成了此次课程设计。

最后的最后，我们再一次感谢所有帮助过我们课程设计的良师益友和同学们。